ENGINEERING CLINICS (ECS)

PROJECT REPORT

SMART PREDICTIVE MAINTENANCE SYSTEM

Project Code: 2003

Submitted by

S. No.	Name	Registration Number
1	Shaik Arshad	25BCE8399
2	Banavath Varun	25BCE8298
3	Kakumanu Mahidhar	25BCE8421
4	Shaik Nadeem Hussain	25BCE8362
5	Marripalepu Hemanth Kumar	25BCE7681
6	Bakka Bharath Ram Teja	25BCE8493

Under the Guidance of

Dr. Nishi Gupta
Assistant Professor Grade-1
School of Advanced Sciences

VIT-AP University

Amaravati, Andhra Pradesh, India

Academic Year 2026–2027

ABSTRACT

This project presents a low-cost embedded monitoring prototype that demonstrates selected principles of condition-based maintenance for a small DC motor. An ESP32 development board acts as the central controller and receives information from three sensor modules: a DHT11 for temperature and relative humidity, an SW-420 module for a digital vibration indication, and an ACS712 20A Hall-effect current sensor for estimating current. A 1.8-inch SPI TFT display provides a local dashboard, while green and red LEDs and a buzzer communicate the system status. The same measurements and status are also sent to the Arduino Serial Monitor for inspection during development and testing.

The firmware samples the sensor channels at intervals of approximately two seconds and compares available measurements with configured warning limits. The current settings are 40 °C for temperature, 80% relative humidity, and 1.5 A for estimated current. A detected vibration or a DHT11 read error also activates the warning state. When no warning condition is detected, the green LED indicates normal status; otherwise, the red LED and buzzer are activated.

The prototype is an educational, threshold-based monitoring system rather than a validated industrial predictive-maintenance solution. The reported current readings varied even when the motor was disconnected, so calibration and independent measurement of the ACS712 channel are essential before interpreting current values. The motor is rated at 12 V, whereas a 9 V battery was used in the reported setup; therefore, the supply arrangement and motor behaviour must be checked. Future development can include calibrated sensing, time-stamped data logging, filtering, fault diagnostics, and predictive models trained and validated on sufficient operating data.

Keywords: ESP32, condition monitoring, predictive maintenance, DHT11, SW-420, ACS712, TFT display, embedded systems.

INDEX

S. No.	Section	Page
1	Abstract	2
2	Index	2
3	List of Figures, List of Tables and Abbreviations	3
4	1. Introduction and 2. Background	4
5	3. Problem Definition, 4. Objectives and 5. Scope and Limitations	5
6	6. Methodology and Procedure	6
7	7. Results and Discussion	10
8	8. Conclusion and Future Scope	14
9	9. References	15
10	Appendix A: Compilation Notes (A.1) and Arduino Source Code (A.2)	15

Page numbers refer to the printed page numbers in the footer of this report.

LIST OF FIGURES

Figure No.	Title	Page
Figure 1	Functional block diagram of the monitoring prototype	6
Figure 2	Hardware assembly of the prototype	7
Figure 3	TFT dashboard photograph	10
Figure 4	Project connection circuit (top view)	11
Figure 5	Project team with guide	12
Figure 6	Illustrative vibration-detected warning state	13

LIST OF TABLES

Table No.	Description	Page
Table 1	Functional stages of the monitoring prototype	6
Table 2	Hardware components and their roles	6
Table 3	ESP32 pin mapping	8
Table 4	Configured warning limits and reported observations	8
Table 5	Functional verification plan	11

ABBREVIATIONS

Abbreviation	Full form / meaning
ADC	Analog-to-Digital Converter
ECS	Engineering Clinics
ESP32	Espressif 32-bit microcontroller family
GPIO	General-Purpose Input/Output
GND	Ground reference
LED	Light-Emitting Diode
SPI	Serial Peripheral Interface
TFT	Thin-Film Transistor display
VCC	Supply voltage

1. INTRODUCTION

Electrical and mechanical equipment can exhibit changes in operating temperature, vibration, and current before a fault becomes obvious to an operator. Observing these variables can help identify conditions that deserve inspection. Maintenance strategies range from corrective maintenance after a failure to preventive maintenance at planned intervals and condition-based approaches that use measurements to guide decisions. Predictive maintenance goes further by using historical and current data to estimate the likelihood or timing of a future fault.

This project demonstrates the basic data-acquisition and alerting layer of such a system. An ESP32 gathers measurements from low-cost sensor modules and applies simple threshold rules. The readings are shown on a small TFT display so that a user can observe the machine condition locally. LEDs and a buzzer provide a quick visual and audible indication when a configured warning condition is detected, and serial output supports debugging while testing.

The prototype is deliberately limited in scope. Its purpose is to integrate sensors, embedded firmware, a display, and warning outputs into one working educational platform. The project does not claim to predict a failure date or diagnose a particular fault. Such claims would require reliable sensor calibration, repeatable experiments, labelled examples of normal and faulty operation, and validation of the predictive method.

2. BACKGROUND AND TECHNICAL CONTEXT

Condition monitoring uses measured variables to understand how equipment is operating. Temperature can indicate excessive heating; vibration can provide evidence of mechanical movement, looseness, or imbalance; and current can change with electrical load or motor operating conditions. No single signal is sufficient in every situation. A current value, for example, depends on the supply, load, motor characteristics, sensor installation, and measurement circuit, so a threshold should be selected from the requirements of the specific equipment and verified through testing.

The ESP32 is suitable for a compact prototype because it combines a microcontroller with digital input/output and analog input capabilities. The DHT11 provides digital temperature and relative-humidity readings. The SW-420 module supplies a digital signal when its vibration circuit detects movement above its sensitivity setting. The ACS712 measures current through a Hall-effect sensing element and produces an analog voltage related to the current. The TFT module presents information in a compact format.

A threshold-based system is a useful first stage because its behaviour is straightforward to explain and test. However, a warning threshold is not itself a prediction. A more advanced predictive-maintenance system would collect time-stamped sensor data over many operating conditions, establish a baseline, extract meaningful features, and validate an anomaly-detection or remaining-useful-life method against independent observations.

3. PROBLEM DEFINITION

A small motor demonstration setup may depend on visual observation to notice unusual heating, vibration, or electrical behaviour. When each variable is checked separately, a user may overlook a change or may not notice that a selected limit has been exceeded. A compact monitoring unit can combine these inputs, show the measured values in one place, and activate an alert when the programmed conditions are met.

The engineering challenge is to integrate different sensor interfaces and ensure that the resulting measurements are interpreted consistently. Digital sensors such as the DHT11 and SW-420 behave differently from an analog current sensor. Wiring errors, supply voltage, grounding, analog-to-digital conversion, electrical noise, sensor placement, and calibration can all affect the values. The current channel is a known limitation in the reported prototype because it showed a non-zero and changing value with the motor disconnected.

Accordingly, the problem addressed by this project is: how can a low-cost ESP32-based prototype collect selected motor-condition signals, display them clearly, and alert a user to threshold or sensor-error conditions, while documenting the limitations that must be resolved before practical deployment?

4. OBJECTIVES OF THE PROPOSED WORK

1. Interface the ESP32 with the DHT11 temperature/humidity module, SW-420 vibration module, and ACS712 current-sensing module.
2. Display temperature, relative humidity, estimated current, vibration state, and overall machine status on a TFT dashboard.
3. Apply configurable warning limits to temperature, humidity, and estimated current.
4. Activate the red LED and buzzer when a warning condition is detected and use the green LED to indicate normal status.
5. Print sensor values and the overall status to the Serial Monitor to support observation and troubleshooting.
6. Document the hardware, pin assignments, firmware sequence, preliminary observations, limitations, and a repeatable verification plan.

5. SCOPE AND LIMITATIONS

The scope includes local sensing, display, threshold comparison, and basic alert outputs for a classroom demonstration with a small DC motor. It does not include a validated machine-learning model, cloud dashboard, automated maintenance scheduling, or a certified industrial safety function. The current display value is an estimate and must not be used as a measurement standard until it has been calibrated against a suitable reference meter.

The project also assumes that the connected hardware is powered and wired safely. A motor must not be powered directly from ESP32 GPIO pins; its supply and any driver must be rated for the motor’s voltage and current. The reported motor rating is 12 V, while the setup used a 9 V battery; that difference may affect motor behaviour and should be resolved during testing.

6. METHODOLOGY AND PROCEDURE

6.1 System Overview

The system follows a simple sensing–processing–display–alert sequence (Table 1, Figure 1). The sensor modules provide measurements or state signals to the ESP32. The controller reads the inputs at a regular interval, checks whether any configured warning rule is true, and updates the display and output indicators. Serial messages provide a second view of the same data for debugging.

Stage	Inputs / operation	Output
Sensing	DHT11 temperature and humidity; SW-420 vibration state; ACS712 analog output	Sensor readings and digital state
Processing	ESP32 samples inputs and compares readings with configured limits	Normal or warning decision
Display	TFT dashboard and Serial Monitor	Human-readable measurements and status
Alert	Green LED, red LED, and buzzer	Visual/audible indication of status

Table 1. Functional stages of the monitoring prototype.

SENSOR INPUTS	CONTROLLER	OUTPUTS
DHT11: temperature / humidity SW-420: vibration state ACS712: current signal	ESP32 DevKit V1 Sampling and input checks Threshold logic	TFT dashboard Serial Monitor Green / red LEDs Buzzer

Figure 1. Functional block diagram of the monitoring prototype.

6.2 Hardware Components

Component	Purpose and notes
ESP32 DevKit V1	Main controller. Reads digital and analog inputs and drives the display and warning outputs.
DHT11 module	Measures temperature and relative humidity; data signal connected to GPIO 4.
SW-420 module	Provides a digital vibration indication; output connected to GPIO 27. Sensitivity depends on module adjustment and mounting.
ACS712 20A module	Hall-effect current sensor. Its analog output is routed through a voltage divider to GPIO 34; calibration is required.
KMR-1.8 SPI TFT	128 × 160 display, described as ST7735-compatible, used to show the dashboard.
Green and red LEDs	Indicate normal and warning status, respectively.
Buzzer	Provides an audible warning when the warning state is active.
DC motor and supply	Motor rated at 12 V; the reported setup used a 9 V battery. Supply compatibility must be verified.

Table 2. Hardware components and their roles.

6.3 Hardware Assembly and Safety

The hardware assembly (Figure 2) combines the ESP32 board, sensor modules, TFT display, LEDs, buzzer, motor, and power wiring. Before energizing the system, inspect the wiring against the pin map, verify common-ground requirements, check for loose connections, and confirm the voltage range of every module. Keep signal wiring separated from high-current motor wiring where practical to reduce interference.

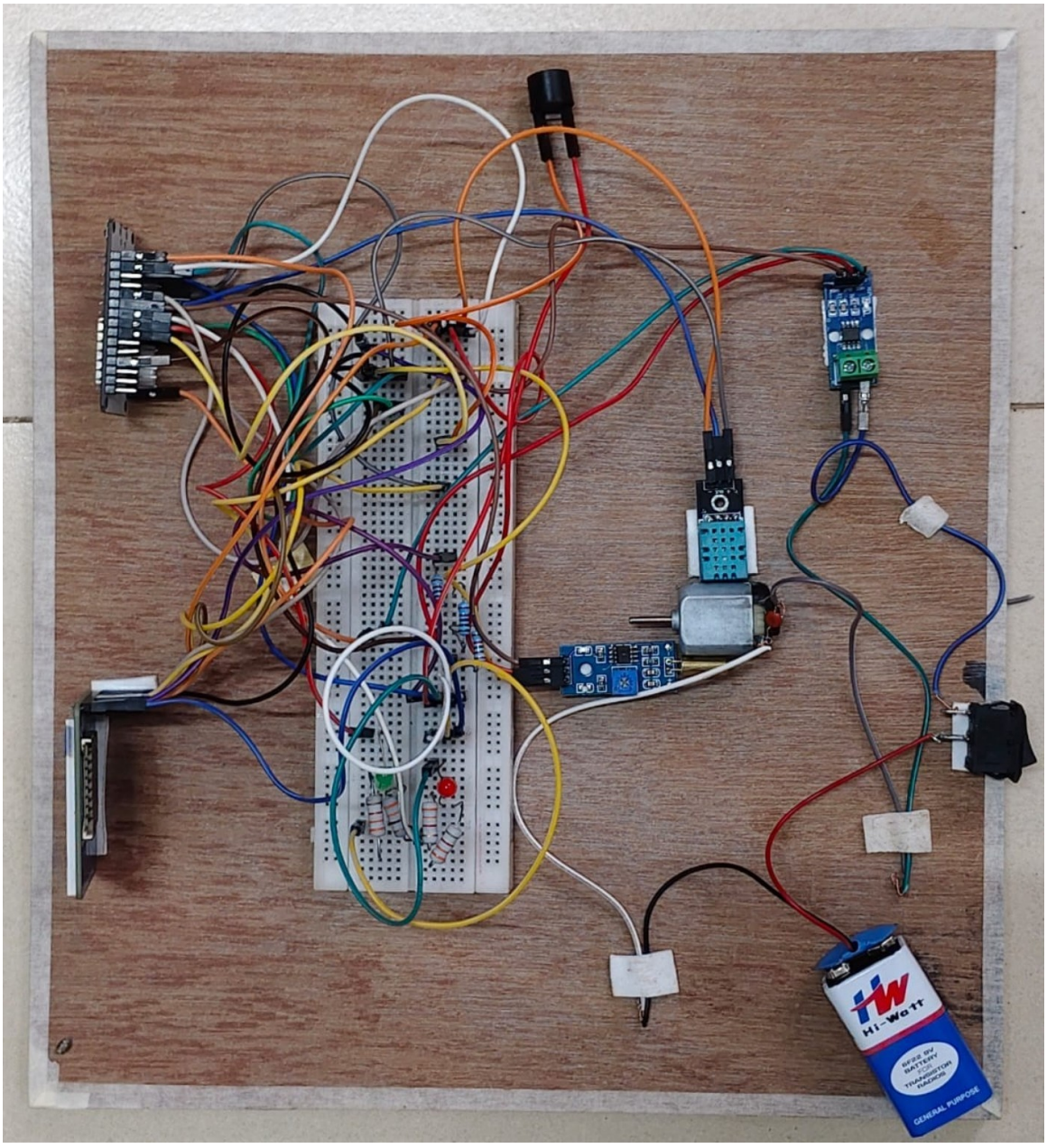


Figure 2. Hardware assembly of the prototype.

Do not connect the motor directly to an ESP32 output pin. Use a suitably rated driver or switching stage where needed, and use a motor supply capable of providing the required current. The motor's stated 12 V rating and the 9 V battery used in the reported assembly should be treated as a test limitation. Disconnect power before changing wiring.

6.4 Pin Mapping

Signal	ESP32 pin / connection
TFT CS / DC / RST	GPIO 5 / GPIO 2 / GPIO 14
TFT SCLK / MOSI	GPIO 18 / GPIO 23
DHT11 data	GPIO 4
SW-420 digital output	GPIO 27
ACS712 divider output	GPIO 34
Green LED / Red LED	GPIO 25 / GPIO 26
Buzzer	GPIO 33

Table 3. ESP32 pin mapping.

All grounds must share an appropriate reference where required by the circuit. Confirm the exact pin labels on the physical modules before connecting power. Ensure the analog voltage reaching GPIO 34 stays within the ESP32 input limits; never assume that a module output is safe without checking its voltage range.

6.5 Firmware Sequence

Step 1. Initialize the Serial Monitor, GPIO directions, ADC resolution and attenuation, DHT11 library, SPI bus, and TFT display.

Step 2. With the motor switched off, sample the current-sensor channel to estimate a zero-current reference voltage.

Step 3. Wait approximately two seconds between measurement cycles.

Step 4. Read temperature and humidity from the DHT11, read the SW-420 digital state, and estimate current from the ACS712 analog channel.

Step 5. Compare valid readings against the configured temperature, humidity, and current limits. Treat a DHT11 read error as a warning.

Step 6. Combine the individual conditions into one warning flag. A detected vibration also activates the warning state.

Step 7. Update the TFT dashboard, print values to the Serial Monitor, and set the LEDs and buzzer according to the warning flag.

6.6 Configured Limits and Reported Observations

Parameter	Reported observation	Configured warning condition
Temperature	Approximately 29–31 °C	Above 40 °C
Relative humidity	Approximately 63–68%	Above 80%
Estimated current	Approximately 0.24–0.38 A; not independently verified	Above 1.5 A
Vibration	Displayed as “Vibrations: No” in the reported normal state	Detected state activates warning
DHT11 status	Valid reading expected during normal operation	Read error activates warning

Table 4. Configured warning limits and reported observations.

These values are preliminary observations and software settings, not certified operating limits. The project report did not include a complete time-stamped test log or independent reference measurements. For a fair evaluation, repeat each test under controlled conditions and record the sensor reading, reference measurement, supply voltage, motor load, and observed indicator state.

6.7 Current-Sensor Calibration

The firmware takes multiple analog samples during startup while the motor is expected to be off. It averages the samples and uses the result as the zero-current reference. During measurement, the ADC voltage is scaled by the configured divider ratio, the zero-current voltage is subtracted, and the difference is divided by the configured ACS712 sensitivity. The absolute value is then displayed as the estimated current.

This approach provides a simple baseline but does not automatically guarantee an accurate current measurement. Accuracy depends on the exact ACS712 variant, sensitivity, divider ratio, ESP32 ADC calibration, wiring, noise, and whether the zero-current condition is stable. Because the reported value changed with the motor disconnected, the current channel should be tested with a known safe load and a suitable reference meter. Confirm the sensor variant and its sensitivity before relying on the configured value.

6.8 Test Procedure

For each test, first establish a safe and stable baseline. Record the actual readings and the status of the LEDs and buzzer. Change only one condition at a time so that the cause of any warning is clear. Avoid exposing electronics to condensation or applying unsafe loads. For current tests, use a known load within the sensor and circuit ratings, and compare the result with an independent meter.

7. RESULTS AND DISCUSSION

The firmware is designed to display temperature, humidity, estimated current, vibration state, and an overall NORMAL or WARNING status (Figure 3). The Serial Monitor repeats these values to assist with debugging. When the warning flag is false, the green LED is switched on and the red LED and buzzer are switched off. When a programmed warning condition is true, the green LED is switched off and the red LED and buzzer are switched on.

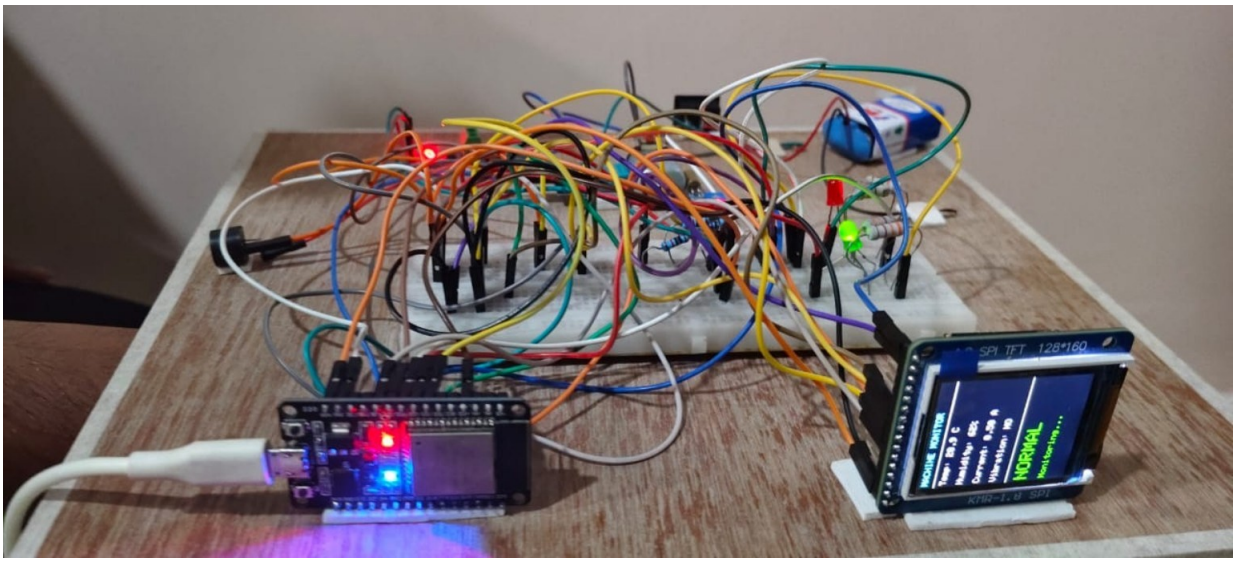


Figure 3. TFT dashboard photograph supplied for the report.

7.1 Reported Observations

The reported setup showed temperature of approximately 29–31 °C, relative humidity of approximately 63–68%, and a nominal estimated current of 0.24–0.38 A. The intended vibration message is “Vibrations: No” when the module does not detect vibration and “Vibrations: Detected” when it does. These values are reported observations rather than independently verified results; the available material does not include a complete test log, uncertainty estimate, or reference-instrument readings.

7.2 Measurement and Implementation Limitations

The most important measurement limitation is the ACS712 channel. During earlier troubleshooting, the current reading was non-zero and changed even with the motor disconnected. Startup offset calibration can reduce a constant offset, but it cannot correct every source of error, including a mismatched sensitivity value, incorrect divider ratio, ADC nonlinearity, electrical noise, grounding problems, or unstable supply conditions. Until calibration is completed, the current display should be described as an estimate.

The DHT11 is a low-cost sensor with limited precision and response speed compared with many industrial sensors. The SW-420 module is a threshold-type vibration detector and does not provide a calibrated vibration spectrum or acceleration measurement. Its response depends on mounting and sensitivity adjustment. Consequently, these modules are suitable for demonstrating the concept but are not sufficient by themselves for industrial fault diagnosis.

7.3 Project Connection Circuit

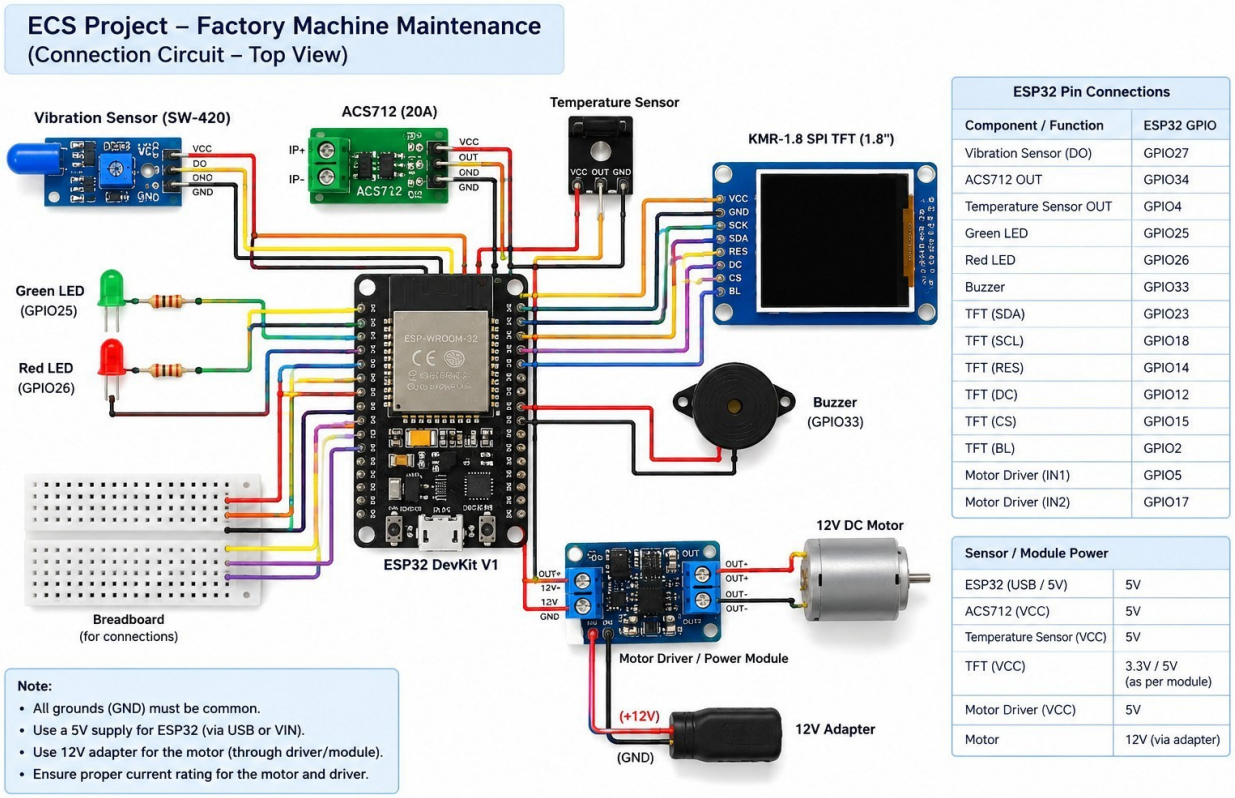


Figure 4. ECS project connection circuit (top view) supplied with the report.

7.4 Functional Verification Plan

Test	Procedure	Expected behaviour
Normal operation	Record readings under a stable condition with no vibration and values below the warning limits.	Measurements are displayed; green LED is on; red LED and buzzer are off.
Temperature warning	Raise sensor temperature in a controlled manner above 40 °C without exceeding component ratings.	Warning state appears; red LED and buzzer activate.
Humidity warning	Expose the sensor to a controlled high-humidity environment above 80%; avoid condensation.	Warning state activates when the valid reading exceeds the limit.
Vibration	Apply a small, controlled vibration at the sensor mount.	Vibration message changes and the warning state activates.
Current warning	Use a known safe load and independently verify current with a suitable meter.	Warning activates if the validated estimated reading exceeds 1.5 A.
Sensor error	Safely simulate a DHT11 read error, such as by disconnecting its data signal with power off before rewiring.	Sensor error is reported and the warning state activates.

Table 5. Functional verification plan.

The table describes expected behaviour for a verification plan. It should not be interpreted as evidence that every test has already been completed. The final experimental report should include the date, test setup, raw readings, reference measurements, pass/fail outcome, and any corrective action for each test.

7.5 Team Documentation



Figure 5. Project team with guide.

The photographs document the project assembly and team activity. They are useful evidence of the prototype’s physical construction, but they do not independently establish measurement accuracy, electrical safety, or successful completion of every test.

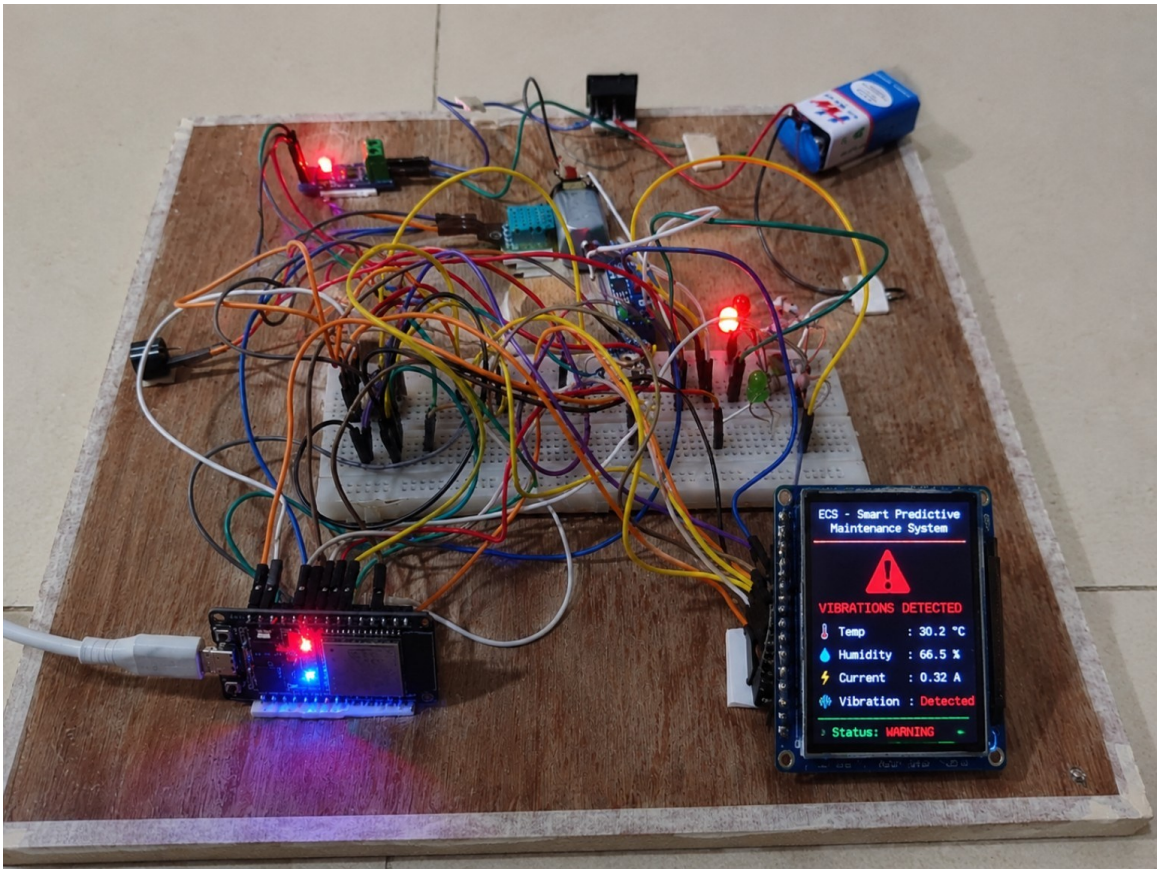


Figure 6. Illustrative vibration-detected warning state.

7.6 Discussion

The prototype demonstrates the integration of sensing, processing, display, and alerting in one embedded platform. Its clearest achievement is the combination of multiple inputs into a common status indicator. The principal next step is measurement validation. Before drawing conclusions about equipment health, the team should calibrate the current channel, verify the supply and motor-driver arrangement, repeat tests, and maintain a structured test log. A reliable predictive-maintenance system would additionally need longer-term data and a validated method for distinguishing normal variation from early fault signatures.

8. CONCLUSION AND FUTURE SCOPE

The Smart Predictive Maintenance System demonstrates a compact embedded monitoring concept using an ESP32, a temperature/humidity sensor, a vibration detector, a current-sensing module, a TFT display, LEDs, and a buzzer. The firmware samples the sensor inputs, compares measurements with configured thresholds, presents a combined status, and provides serial output for troubleshooting. These features make the prototype useful for learning sensor interfacing, analog input handling, display programming, and basic condition-monitoring logic.

The current implementation should be regarded as a threshold-based educational prototype rather than a validated predictive-maintenance product. The ACS712 channel requires calibration and independent verification, and the reported motor/supply combination needs review. Repeatable tests are also needed to confirm that the warning logic, display, and output indicators behave as intended. Predictive claims should only be made after adequate labelled data have been collected and a suitable predictive method has been evaluated.

8.1 Future Scope

1. Calibrate the ACS712 channel against a suitable reference meter and known loads, confirming the exact sensor sensitivity and voltage-divider ratio.
2. Use a motor supply and driver that are correctly rated for the motor’s voltage and current, with appropriate protection.
3. Record time-stamped sensor values under normal operation and controlled abnormal conditions.
4. Add filtering, averaging, vibration debounce logic, and sensor-fault diagnostics to improve stability.
5. Create a health indicator or anomaly-detection model only after collecting sufficient representative data and validating it on separate test data.
6. Add data logging or Wi-Fi-based monitoring after local measurements and alerts have been verified.
7. Improve the test protocol by documenting raw measurements, calibration steps, environmental conditions, and pass/fail outcomes.

9. REFERENCES

[1] Espressif Systems. ESP32 Series Datasheets and Technical Reference Documentation.

[2] Arduino. Arduino Language Reference, including analogRead and analog input documentation.

[3] Adafruit. DHT Sensor Library and Adafruit GFX Library documentation.

[4] Adafruit. ST7735/ST7789 display library documentation.

[5] Allegro MicroSystems. ACS712 Hall-Effect-Based Linear Current Sensor IC datasheet.

[6] SW-420 vibration sensor module documentation supplied by the module manufacturer or vendor.

[7] STMicroelectronics. ST7735 display controller documentation.

The references above identify the documentation families used for the prototype. For a final academic submission, add the exact document titles, revision numbers, access dates, and official URLs for the specific versions consulted.

APPENDIX A: ARDUINO SOURCE CODE

Appendix A.2 reproduces the ESP32 firmware described in this report on a single page in two columns. It uses the SPI, Adafruit GFX, Adafruit ST7735, DHT, and math libraries. Confirm that the required libraries are installed and that the display controller, pin mapping, current-sensor variant, divider ratio, and electrical limits match the physical hardware before uploading the sketch.

A.1 Notes for Compilation and Testing

1. Install the Adafruit GFX, Adafruit ST7735, and DHT sensor libraries in the Arduino IDE.
2. Select the correct ESP32 board and serial port, then compile before connecting the motor supply.
3. Verify that the display initialization option and rotation match the exact TFT module.
4. Confirm the ACS712 variant and sensitivity. The code’s 0.100 V/A setting and divider ratio are configuration assumptions and must be checked against the actual circuit.
5. Keep the motor off during startup calibration. A changing or unstable zero-current reference means the current readings are not yet trustworthy.
6. Test alert outputs with safe, controlled conditions and document measured results. Do not treat the warning limits as industrial safety limits.

A.2 Complete Sketch – Arduino Source Code (215 lines)

```
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7735.h>
#include <DHT.h>
#include <math.h>

#define TFT_CS   5
#define TFT_DC   2
#define TFT_RST 14
#define TFT_SCLK 18
#define TFT_MOSI 23

Adafruit_ST7735 tft(TFT_CS, TFT_DC, TFT_RST);

#define DHT_PIN       4
#define DHT_TYPE      DHT11
#define VIBRATION_PIN 27
#define CURRENT_PIN   34
#define GREEN_LED     25
#define RED_LED       26
#define BUZZER_PIN    33

DHT dht(DHT_PIN, DHT_TYPE);

const float TEMP_LIMIT = 40.0;
const float HUMIDITY_LIMIT = 80.0;
const float CURRENT_LIMIT = 1.5;
const float ACS_SENSITIVITY = 0.100;
const float DIVIDER_RATIO = 1.5;
const bool VIBRATION_ACTIVE_HIGH = true;

float temperature = NAN, humidity = NAN;
float currentAmps = 0.0, zeroCurrentVoltage = 2.50;
bool vibrationDetected = false, warning = false;
unsigned long lastRead = 0;
const unsigned long READ_INTERVAL = 2000;

float readCurrent() {
  uint32_t totalMilliVolts = 0;
  for (int i = 0; i < 20; i++) {
    totalMilliVolts += analogReadMilliVolts(CURRENT_PIN);
    delay(2);
  }

  float adcVoltage = (totalMilliVolts / 20.0) / 1000.0;
  float sensorVoltage = adcVoltage * DIVIDER_RATIO;
  float amps = (sensorVoltage - zeroCurrentVoltage)
               / ACS_SENSITIVITY;
    return fabs(amps);
}

void calibrateCurrentSensor() {
  Serial.println("Calibrating ACS712...");
  Serial.println("Keep motor OFF.");
  delay(1500);

    uint32_t totalMilliVolts = 0;
    for (int i = 0; i < 100; i++) {
      totalMilliVolts += analogReadMilliVolts(CURRENT_PIN);
      delay(10);
    }

    float adcVoltage = (totalMilliVolts / 100.0) / 1000.0;
    zeroCurrentVoltage = adcVoltage * DIVIDER_RATIO;
    Serial.print("Zero-current voltage: ");
    Serial.println(zeroCurrentVoltage, 3);
}

void displayDashboard() {
  tft.fillScreen(ST77XX_BLACK);
  tft.setTextWrap(false);
  tft.setTextSize(1);
  tft.setTextColor(ST77XX_CYAN);
  tft.setCursor(4, 4);
  tft.println("MACHINE MONITOR");
  tft.drawFastHLine(0, 16, 160, ST77XX_WHITE);

    tft.setTextColor(ST77XX_WHITE);
    tft.setCursor(4, 24);
    tft.print("Temp: ");
    if (isnan(temperature)) {
      tft.println("Sensor error");
    } else {
      tft.print(temperature, 1);
      tft.println(" C");
    }

    tft.setCursor(4, 42);
    tft.print("Humidity: ");
    if (isnan(humidity)) {
      tft.println("Sensor error");
    } else {
      tft.print(humidity, 0);
      tft.println(" %");
    }

    tft.setCursor(4, 60);
    tft.print("Current: ");
    tft.print(currentAmps, 2);
    tft.println(" A");

    tft.setCursor(4, 78);
    tft.print("Vibrations: ");
    tft.println(vibrationDetected ? "Detected" : "No");
    tft.drawFastHLine(0, 94, 160, ST77XX_WHITE);

    if (warning) {
      tft.setTextColor(ST77XX_RED);
      tft.setTextSize(2);
      tft.setCursor(4, 102);
      tft.println("WARNING!");
    } else {
      tft.setTextColor(ST77XX_GREEN);
      tft.setTextSize(2);
      tft.setCursor(4, 102);
      tft.println("NORMAL");
    }
}

void printSerialData() {
  Serial.println("\\n===== MACHINE STATUS =====");
  Serial.print("Temperature: ");
  if (isnan(temperature)) {
    Serial.println("DHT11 read error");
  } else {
    Serial.print(temperature, 1);
    Serial.println(" C");
  }

    Serial.print("Humidity: ");
    if (isnan(humidity)) {
      Serial.println("DHT11 read error");
    } else {
      Serial.print(humidity, 1);
      Serial.println(" %");
    }

    Serial.print("Current: ");
    Serial.print(currentAmps, 2);
    Serial.println(" A");

    Serial.print("Vibrations: ");
    Serial.println(vibrationDetected ? "Detected" : "No");
    Serial.print("Machine status: ");
    Serial.println(warning ? "WARNING" : "NORMAL");
    Serial.println("==========================");
}

void setup() {
  Serial.begin(115200);
  pinMode(VIBRATION_PIN, INPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER_PIN, LOW);

    analogReadResolution(12);
    analogSetPinAttenuation(CURRENT_PIN, ADC_11db);
    dht.begin();

    SPI.begin(TFT_SCLK, -1, TFT_MOSI, TFT_CS);
    tft.initR(INITR_GREENTAB);
    tft.setRotation(1);
    tft.fillScreen(ST77XX_BLACK);
    tft.setTextColor(ST77XX_WHITE);
    tft.setTextSize(1);
    tft.setCursor(4, 30);
    tft.println("Machine Monitor");
    tft.setCursor(4, 50);
    tft.println("Starting...");

    calibrateCurrentSensor();
    Serial.println("\\nSmart Predictive Maintenance");
    Serial.println("System started.");
    delay(1000);
}

void loop() {
  if (millis() - lastRead < READ_INTERVAL) {
    return;
  }
  lastRead = millis();

    temperature = dht.readTemperature();
    humidity = dht.readHumidity();

    int vibrationState = digitalRead(VIBRATION_PIN);
    if (VIBRATION_ACTIVE_HIGH) {
      vibrationDetected = (vibrationState == HIGH);
    } else {
      vibrationDetected = (vibrationState == LOW);
    }

    currentAmps = readCurrent();

    bool temperatureWarning =
        !isnan(temperature) && temperature > TEMP_LIMIT;
    bool humidityWarning =
        !isnan(humidity) && humidity > HUMIDITY_LIMIT;
    bool currentWarning = currentAmps > CURRENT_LIMIT;
    bool sensorWarning = isnan(temperature) || isnan(humidity);

    warning = vibrationDetected || temperatureWarning ||
              humidityWarning || currentWarning || sensorWarning;

    digitalWrite(GREEN_LED, warning ? LOW : HIGH);
    digitalWrite(RED_LED, warning ? HIGH : LOW);
    digitalWrite(BUZZER_PIN, warning ? HIGH : LOW);

    displayDashboard();
    printSerialData();
}
```